

Quiz 2

315832025 連怡卉

AI Tools: Gemini

1. 直方圖均衡化的核心數學原理 📊

直方圖均衡化的本質是建立一個查表映射 (Lookup Table, LUT)，把原本擠在一起的舊灰階值 r_k 重新對應到新的灰階值 s_k 。

轉換步驟如下：

- 計算直方圖 (Histogram)：統計灰階值 $0 \sim 255$ 每個值出現的次數 n_k 。
- 計算機率密度 (PDF)： $p(r_k) = \frac{n_k}{M \times N}$ (其中 $M \times N$ 為總像素數)。
- 計算累積分布函數 (CDF)：

$$\text{cdf}(k) = \sum_{j=0}^k p(r_j)$$

CDF 的值介於 $0 \sim 1$ 之間，且保證單調遞增。

- 線性縮放映射：將累積機率拉伸到 $0 \sim 255$ ：

$$s_k = \text{round} \left(\frac{\text{cdf}(k) - \text{cdf}_{\min}}{1 - \text{cdf}_{\min}} \times 255 \right)$$

(若直接以最簡式表示，亦常寫為 $s_k = \text{round}(\text{cdf}(k) \times 255)$)

2. 演算法實作：Code/histogram.py 📄

在 Code/ 目錄建立 histogram.py，放入我們自行用 NumPy 實作的均衡化函式：

```
Python

import numpy as np

def equalize_hist_numpy(gray_img: np.ndarray) -> np.ndarray:
    """
    使用 NumPy 自行實作的灰階直方圖均衡化
    輸入必須為單通道 uint8 灰階影像
    """
    # 1. 統計直方圖 (計算每個灰階值 0~255 出現次數)
    hist, _ = np.histogram(gray_img.flatten(), bins=256, range=[0, 256])

    # 2. 計算累積分布函數 (CDF)
    cdf = hist.cumsum()

    # 3. 遮罩排除次數為 0 的像素，並使用正規化公式映射到 0~255
    cdf_mask = np.ma.masked_equal(cdf, 0)
    cdf_mask = (cdf_mask - cdf_mask.min()) * 255 / (cdf_mask.max() - cdf_mask.min())
    cdf_final = np.ma.filled(cdf_mask, 0).astype('uint8')

    # 4. 利用查表 (LUT) 快速對每個像素做映射
    return cdf_final[gray_img]
```

3. 主實驗流程：在 main.ipynb 測試與比較 🚀

在 Notebook 裡調用 OpenCV 官方函式 `cv2.equalizeHist()` 與我們剛寫好的 `equalize_hist_numpy()`：

```
Python

import cv2
import matplotlib.pyplot as plt
import numpy as np
from Code.histogram import equalize_hist_numpy

# 1. 讀取灰階影像
gray = cv2.imread('Data/nightmarket.jpg', cv2.IMREAD_GRAYSCALE)

# 2. 執行兩種方法
equ_cv = cv2.equalizeHist(gray)
equ_np = equalize_hist_numpy(gray)

# 3. 繪製對比圖與直方圖
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

# 原圖與其直方圖
axes[0, 0].imshow(gray, cmap='gray')
axes[0, 0].set_title('Original Gray')
axes[0, 0].axis('off')
axes[0, 1].hist(gray.ravel(), bins=256, range=[0, 256], color='black')
axes[0, 1].set_title('Original Histogram')

# 均衡化後影像與其直方圖
axes[1, 0].imshow(equ_np, cmap='gray')
axes[1, 0].set_title('Equalized (NumPy)')
axes[1, 0].axis('off')
axes[1, 1].hist(equ_np.ravel(), bins=256, range=[0, 256], color='black')
axes[1, 1].set_title('Equalized Histogram')

plt.tight_layout()
plt.show()
```

我們來把基礎題的呈現方式一次整理好。

題目對基礎題的要求是：

1. 影像均衡化前後對比。
2. 繪製均衡化處理前後的灰階 Histogram (直方圖)。

為了像「表格」一樣整齊排列，最常見的呈現是 2 列 2 行 (2 × 2) 的網格版面：

- 第一列 (上)：原圖  vs. 均衡化後影像 
- 第二列 (下)：原圖直方圖  vs. 均衡化後直方圖 

在 `Code/plot_utils.py` 新增表格化呈現函式

可以在 `Code/plot_utils.py` 加入一個專門用來排版影像與直方圖的函式：

```
Python

import matplotlib.pyplot as plt

def plot_image_and_hist_grid(img_before, img_after, title_before="Before", title_after="After"):
    """
    以 2x2 表格網格呈現影像與直方圖的對比
    """
    fig, axes = plt.subplots(2, 2, figsize=(12, 8))

    # 1. 影像部分 (第一列)
    axes[0, 0].imshow(img_before, cmap='gray')
    axes[0, 0].set_title(f"Image: {title_before}")
    axes[0, 0].axis('off')

    axes[0, 1].imshow(img_after, cmap='gray')
    axes[0, 1].set_title(f"Image: {title_after}")
    axes[0, 1].axis('off')

    # 2. 直方圖部分 (第二列)
    # ravel() 將二維矩陣扁平為一維，以便繪製頻率分佈
    axes[1, 0].hist(img_before.ravel(), bins=256, range=[0, 256], color='black')
    axes[1, 0].set_title(f"Histogram: {title_before}")
    axes[1, 0].set_xlim([0, 256])

    axes[1, 1].hist(img_after.ravel(), bins=256, range=[0, 256], color='black')
    axes[1, 1].set_title(f"Histogram: {title_after}")
    axes[1, 1].set_xlim([0, 256])

    plt.tight_layout()
    plt.show()
```

在 `main.ipynb` 呼叫執行

儲存好 `plot_utils.py` 後，在 Notebook 基礎題的區塊只要寫這幾行即可：

```
Python

import cv2
from Code.plot_utils import plot_image_and_hist_grid

# 讀取並執行均衡化
img = cv2.imread('Data/beach.jpg', cv2.IMREAD_GRAYSCALE)
equ_cv = cv2.equalizeHist(img)

# 以 2x2 表格呈現
plot_image_and_hist_grid(img, equ_cv, title_before="Original", title_after="OpenCV Equalized")
```

把這段執行起來後，觀察下方的兩個直方圖：原本擠在一起的數值分佈，在經過 `cv2.equalizeHist` 處理後，形狀產生了什麼變化呢？

就像調色/光一樣，把兩邊的極大值拉到了該before圖的左右兩側

這個觀察非常精準！原本擁擠在某個區間的像素值，被硬生生往兩側拉伸擴展到了整個 0 ~ 255 的範圍，這正是提升對比度的核心效果。

完成了基礎部分後，我們接著進入 Quiz 2 的進階部分：

- 使用 NumPy 自行實作（我們先前寫在 `Code/histogram.py` 的 `equalize_hist_numpy`）。
- 重複執行 N 次測量時間（比較 OpenCV 與 NumPy 的執行速度）。
- 比較兩種方法之間的差異（如 MAE、最大差值）。

在寫進階比對程式時，延續 Quiz 1 的經驗：

你打算讓 N 設定為多少次來測速？另外，為了驗證我們寫的 NumPy 均衡化與 OpenCV 的效果是否一致，你想先看執行速度比對，還是計算兩張圖的差值 (MAE) 呢？

👤 💬 📄 ...

(1) 執行速度測試 🐢

我們設定 $N = 100$ ，分別對 `cv2.equalizeHist` 與 `equalize_hist_numpy` 測量平均單次執行時間：

```
Python

import time

N = 100

# 測試 OpenCV
start = time.perf_counter()
for _ in range(N):
    _ = cv2.equalizeHist(img)
time_cv = (time.perf_counter() - start) / N

# 測試 NumPy
start = time.perf_counter()
for _ in range(N):
    _ = equalize_hist_numpy(img)
time_np = (time.perf_counter() - start) / N

print(f"OpenCV 平均執行時間: {time_cv * 1000:.4f} ms")
print(f"NumPy 平均執行時間: {time_np * 1000:.4f} ms")
print(f"速度差距: NumPy 約為 OpenCV 的 {time_np / time_cv:.2f} 倍慢")
```

(2) 影像增強效果 🖼️

將 NumPy 與 OpenCV 均衡化後的影像與直方圖並排展示，觀察兩者在視覺對比與細節還原上的表現：

```
Python

from Code.plot_utils import plot_image_and_hist_grid

# 取得兩者結果
equ_np = equalize_hist_numpy(img)

# 並排比較兩者的影像與直方圖
plot_image_and_hist_grid(equ_cv, equ_np, title_before="OpenCV", title_after="NumPy")
```

(3) 兩種方法之間的差異 📊

延續 Quiz 1 的量化分析方式，計算絕對差值並找出最大差異與平均誤差：

```
Python

# 計算差異矩陣 (轉成 int 避免 uint8 溢位)
diff_hist = np.abs(equ_cv.astype(int) - equ_np.astype(int))

max_diff = np.max(diff_hist)
mae = np.mean(diff_hist)

print(f"最大像素差值: {max_diff}")
print(f"平均絕對誤差 (MAE): {mae:.4f}")
```

將這三段程式碼放入 Notebook 執行後：

- 測得的執行時間中，NumPy 大約比 OpenCV 慢了幾倍？
- 兩者計算出來的 `max_diff` 與 `mae` 數值是多少？

```
OpenCV 平均執行時間: 0.8178 msNumPy 平均執行時間:
6.5844 ms
最大像素差值: 1平均絕對誤差 (MAE): 0.5457
```

這組實驗數據非常清晰且合理。

我們來分析這幾個關鍵結果：

- 🐢 執行速度：NumPy 大約花了 6.58 ms，OpenCV 只要 0.82 ms，OpenCV 快了約 8 倍。這展現了 OpenCV 底層使用 C++ 與硬體指令集（如 SIMD/AVX）優化的強大效能。
- 📊 數值差異：
 - 最大像素差值只有 1，代表我們自己推導的 CDF 映射演算法在數學邏輯上是完全正確的。
 - MAE 約為 0.55，說明有將近一半的像素產生了 ± 1 的微小偏差。這主要來自浮點數轉整數時的捨入方式（Rounding vs. Truncation）差異。

這樣一來，Quiz 2 的要求就全部達成了！